

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 2

«Вычисление яркости в точках на поверхности»

Выполнили студенты:

Баянов Равиль Динарович

Кузнецов Даниил Александрович

Группа № Р3434

Преподаватель:

Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2025

Цель

Изучить, как вычисляется яркость точек на плоскости треугольника с учетом освещения от точечных источников света и диффузно отражательных свойств поверхности.

Используемые формулы

1. «Цветная» интенсивность источника под углом к оси источника света:

$I(RGB, \vec{s}) = I_0(RGB) \cos \theta$, где $I_0(RGB)$ – «цветная» интенсивность источника света в направлении его оси $\vec{O}$, $\|\vec{O}\| = 1$, θ - угол между направлением распространения света и осью источника света, $\cos \theta$ – диаграмма излучения.

2. «Цветная» освещенность точки:

$E(RGB, \vec{P}_T) = \frac{I(RGB, \vec{s}) \cos \alpha}{R^2}$, где $I(RGB, \vec{s})$ – «цветная» интенсивность света, α - угол между направлением света и нормалью к освещаемой поверхности, R - расстояние от источника света до рассматриваемой точки.

3. Перевод локальных координат точки в плоскости в глобальные

$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y \right)$, где x и y смещения по ребрам треугольника.

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{(\vec{P}_2 - \vec{P}_0) \times (\vec{P}_1 - \vec{P}_0)}{\|(\vec{P}_1 - \vec{P}_0) \times (\vec{P}_2 - \vec{P}_0)\|}$$

5. Вектор от точки плоскости до источника света:

$$\vec{s} = \vec{P}_T - \vec{P}_L, R^2 = \|\vec{s}\|^2, \cos \alpha = \frac{\vec{s} \cdot \vec{N}}{\|\vec{s}\|}, \cos \theta = \frac{\vec{s} \cdot \vec{O}}{\|\vec{s}\|}$$

6. Яркость точки:

$$L(RGB, \vec{P}_T, \vec{v}) = \frac{1}{\pi} \sum_{i \in lights} \left(E_i(RGB, \vec{P}_T) \cdot f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i) \right), \quad \text{где}$$

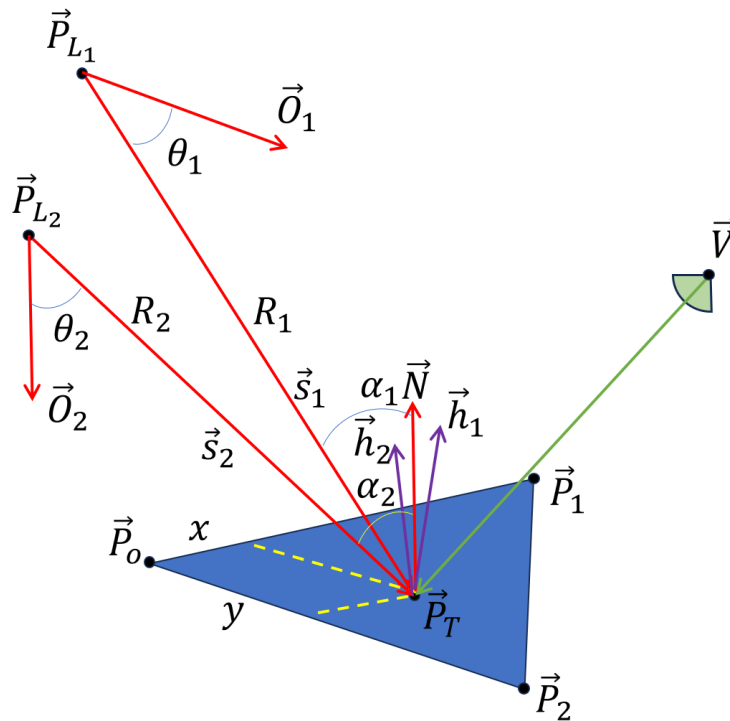
$E_i(RGB, \vec{P}_T)$ - освещённость от i -го источника света, $\vec{P}_T$ - координаты точки, $\vec{v}$ - направление на наблюдения, $\vec{s}_i$ - направление на источник света, $f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i)$ - двунаправленная функция отражения (BRDF), $K(RGB)$ – «цвет» поверхности.

7. Двунаправленная функция отражения

$f(RGB, \vec{P}_T, \vec{v}, \vec{s}_i) = K(RGB) \left(k_d + k_s (\vec{h} \cdot \vec{N})^{k_e} \right)$, где k_d - коэффициент диффузного отражения поверхности, k_s - коэффициент «зеркальности» поверхности, k_e - коэффициент, определяющий ширину блика, $\vec{h}$ - средний вектор между направлениями освещения и наблюдения, $\vec{N}$ - нормаль к поверхности в точке освещения.

8. Средний вектор

$$\vec{h} = \frac{\vec{v} + \vec{s}}{\|\vec{v} + \vec{s}\|}.$$



Шаги алгоритма.

1. Для каждой расчётной точки (x, y) перевести локальные координаты в глобальные
2. По трём точкам треугольника вычислить нормаль к её плоскости
3. Вычисление параметров освещения для точки
 - Вектор до источника света
 - Расстояние
 - Углы $\cos(\alpha)$ между нормалью и направлением света, $\cos(\theta)$ между векторами источника и направлением света
 - Интенсивность направления
 - Освещённость
4. Вычисление BRDF-функции для отражения
5. Расчет итоговой яркости

Программа для расчета яркости

```
#include <stdio.h>
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

#define INV_PI (1.0 / M_PI)

// Структуры
typedef struct {
    double R;
    double G;
    double B;
} RGBIntensity;

typedef struct {
    double x;
    double y;
    double z;
} Vector3D;

typedef struct {
    Vector3D s; // Вектор от точки к источнику
    double R_squared; // Квадрат расстояния (R^2)
    double cos_alpha; // Косинус угла падения (между N и S)
    double cos_theta; // Косинус угла прожектора (между direction_of_spot
и direction_from_source_to_point)
```

```

} LightParameters;

// ----- Векторные операции -----
Vector3D vector_subtract(Vector3D A, Vector3D B) {
    Vector3D result = {A.x - B.x, A.y - B.y, A.z - B.z};
    return result;
}

double vector_length(Vector3D V) {
    return sqrt(V.x * V.x + V.y * V.y + V.z * V.z);
}

Vector3D vector_scale(Vector3D V, double scalar) {
    Vector3D result = {V.x * scalar, V.y * scalar, V.z * scalar};
    return result;
}

Vector3D vector_add(Vector3D A, Vector3D B) {
    Vector3D result = {A.x + B.x, A.y + B.y, A.z + B.z};
    return result;
}

Vector3D vector_cross(Vector3D A, Vector3D B) {
    Vector3D result = {
        A.y * B.z - A.z * B.y,
        A.z * B.x - A.x * B.z,
        A.x * B.y - A.y * B.x
    };
    return result;
}

double vector_dot(Vector3D A, Vector3D B) {
    return A.x * B.x + A.y * B.y + A.z * B.z;
}

Vector3D vector_normalize(Vector3D V) {
    double len = vector_length(V);
    if (len > 1e-12) {
        return vector_scale(V, 1.0 / len);
    } else {
        return (Vector3D){0.0, 0.0, 0.0};
    }
}

// ----- RGB операции -----
RGBIntensity rgb_multiply(RGBIntensity A, RGBIntensity B) {
    RGBIntensity result;
    result.R = A.R * B.R;
    result.G = A.G * B.G;
    result.B = A.B * B.B;
    return result;
}

RGBIntensity rgb_add(RGBIntensity A, RGBIntensity B) {
    RGBIntensity result = {A.R + B.R, A.G + B.G, A.B + B.B};
    return result;
}

RGBIntensity rgb_scale(RGBIntensity V, double scalar) {
    RGBIntensity result = {V.R * scalar, V.G * scalar, V.B * scalar};
    return result;
}

```

```

}

// ----- Освещение -----
RGBIntensity calculate_directional_intensity(RGBIntensity I0_RGB, double
cos_theta) {
    return rgb_scale(I0_RGB, fmax(0.0, cos_theta));
}

// Цветная освещённость точки (E) от одного источника
RGBIntensity calculate_color_illuminance(RGBIntensity I_RGB_directional,
double cos_alpha, double R_squared) {
    if (R_squared <= 1e-12) {
        return (RGBIntensity){0.0, 0.0, 0.0};
    }
    double factor = fmax(0.0, cos_alpha) / R_squared;
    return rgb_scale(I_RGB_directional, factor);
}

// ----- Геометрия -----
Vector3D local_to_global_coords(Vector3D P0, Vector3D P1, Vector3D P2,
double local_x, double local_y) {
    Vector3D V10 = vector_subtract(P1, P0);
    Vector3D V20 = vector_subtract(P2, P0);
    Vector3D Displacement1 = vector_scale(V10, local_x);
    Vector3D Displacement2 = vector_scale(V20, local_y);
    Vector3D TotalDisplacement = vector_add(Displacement1,
Displacement2);
    Vector3D PT = vector_add(P0, TotalDisplacement);
    return PT;
}

// Нормаль треугольника (единичная)
Vector3D calculate_triangle_normal(Vector3D P0, Vector3D P1, Vector3D P2)
{
    Vector3D A = vector_subtract(P1, P0);
    Vector3D B = vector_subtract(P2, P0);
    Vector3D N_unnormalized = vector_cross(A, B);
    return vector_normalize(N_unnormalized);
}

// Центроид треугольника
Vector3D triangle_centroid(Vector3D P0, Vector3D P1, Vector3D P2) {
    Vector3D sum = vector_add(P0, vector_add(P1, P2));
    return vector_scale(sum, 1.0 / 3.0);
}

// Параметры света
LightParameters calculate_light_params(Vector3D P_T, Vector3D P_L,
Vector3D N_plane, Vector3D O_spot) {
    LightParameters params;
    params.s = vector_subtract(P_L, P_T);
    params.R_squared = vector_dot(params.s, params.s);

    if (params.R_squared < 1e-24) {
        params.cos_alpha = 0.0;
        params.cos_theta = 0.0;
        return params;
    }

    Vector3D s_normalized = vector_normalize(params.s);
    Vector3D O_normalized = vector_normalize(O_spot);

```

```

        params.cos_alpha = vector_dot(s_normalized, N_plane);
        params.cos_theta = -vector_dot(s_normalized, O_normalized);

    return params;
}

// ----- BRDF -----
RGBIntensity calculate_brdf_blinn_phong(RGBIntensity K_RGB, double kd,
double ks, double ke, Vector3D h, Vector3D N) {
    double NdH = fmax(0.0, vector_dot(N, h));
    double diffuse = kd * INV_PI;
    double spec = ks * ((ke + 8.0) * INV_PI * 0.125) * pow(NdH, ke);
    return rgb_scale(K_RGB, diffuse + spec);
}

Vector3D calculate_halfway_vector(Vector3D v, Vector3D s) {
    Vector3D V_sum = vector_add(v, s);
    return vector_normalize(V_sum);
}

// ----- Функции высокого уровня -----
// Освещённость от одного источника (для таблиц E1/E2)
RGBIntensity calculate_E_RGB_PT(
    Vector3D P0, Vector3D P1, Vector3D P2,
    double local_x, double local_y,
    RGBIntensity I0_RGB,
    Vector3D PL, Vector3D O_spot)
{
    Vector3D PT = local_to_global_coords(P0, P1, P2, local_x, local_y);
    Vector3D N = calculate_triangle_normal(P0, P1, P2);
    if (vector_length(N) < 1e-12) {
        return (RGBIntensity){0.0, 0.0, 0.0};
    }

    LightParameters params = calculate_light_params(PT, PL, N, O_spot);
    RGBIntensity I_dir = calculate_directional_intensity(I0_RGB,
params.cos_theta);
    return calculate_color_illuminance(I_dir, params.cos_alpha,
params.R_squared);
}

// Яркость в направлении камеры с учётом двух источников
RGBIntensity calculate_L_RGB_PT(
    Vector3D P0, Vector3D P1, Vector3D P2,
    double local_x, double local_y,
    RGBIntensity I0_1_RGB, RGBIntensity I0_2_RGB,
    Vector3D PL1, Vector3D PL2,
    Vector3D O1_spot, Vector3D O2_spot,
    Vector3D V_camera,
    RGBIntensity K_RGB,
    double kd, double ks, double ke)
{
    Vector3D PT = local_to_global_coords(P0, P1, P2, local_x, local_y);
    Vector3D N = calculate_triangle_normal(P0, P1, P2);
    if (vector_length(N) < 1e-12) {
        return (RGBIntensity){0.0, 0.0, 0.0};
    }

    Vector3D V_cam_n = vector_normalize(V_camera);
    if (vector_dot(N, V_cam_n) <= 1e-12) {
        return (RGBIntensity){0.0, 0.0, 0.0};
    }
}

```

```

    }

    RGBIntensity total = {0.0, 0.0, 0.0};
    if (IO_1_RGB.R != 0.0 || IO_1_RGB.G != 0.0 || IO_1_RGB.B != 0.0) {
        LightParameters p1 = calculate_light_params(PT, PL1, N, O1_spot);
        Vector3D S1_n = vector_normalize(p1.s);
        RGBIntensity I1_dir = calculate_directional_intensity(IO_1_RGB,
p1.cos_theta);
        double f1 = fmax(0.0, p1.cos_alpha) / p1.R_squared;
        RGBIntensity E1 = rgb_scale(I1_dir, f1);
        Vector3D H1 = calculate_halfway_vector(V_cam_n, S1_n);
        RGBIntensity BRDF1 = calculate_brdf_blinn_phong(K_RGB, kd, ks,
ke, H1, N);
        total = rgb_add(total, rgb_multiply(E1, BRDF1));
    }

    if (IO_2_RGB.R != 0.0 || IO_2_RGB.G != 0.0 || IO_2_RGB.B != 0.0) {
        LightParameters p2 = calculate_light_params(PT, PL2, N, O2_spot);
        Vector3D S2_n = vector_normalize(p2.s);
        RGBIntensity I2_dir = calculate_directional_intensity(IO_2_RGB,
p2.cos_theta);
        double f2 = fmax(0.0, p2.cos_alpha) / p2.R_squared;
        RGBIntensity E2 = rgb_scale(I2_dir, f2);
        Vector3D H2 = calculate_halfway_vector(V_cam_n, S2_n);
        RGBIntensity BRDF2 = calculate_brdf_blinn_phong(K_RGB, kd, ks,
ke, H2, N);
        total = rgb_add(total, rgb_multiply(E2, BRDF2));
    }

    return total;
}

int main() {
    Vector3D V_camera_static = {0.0, 0.0, 1.0};
    RGBIntensity K_RGB = {0.7, 0.7, 0.1};
    double kd = 0.8;
    double ks = 0.2;
    double ke = 10.0;

    Vector3D P0 = {0.0, 0.0, 0.0};
    Vector3D P1 = {1.0, 0.0, 0.0};
    Vector3D P2 = {0.0, 1.0, 0.0};
    Vector3D C = triangle_centroid(P0, P1, P2);

    RGBIntensity IO_1_RGB = {8, 6, 4};
    RGBIntensity IO_2_RGB = {3, 5, 7};

    Vector3D PL1 = {0.20, 0.85, 0.60};
    Vector3D PL2 = {0.85, 0.20, 0.70};
    Vector3D O1_spot = {0.166055, -0.643464, -0.747248};
    Vector3D O2_spot = {-0.586999, 0.151484, -0.795289};

    double x_vals[5] = {0.1, 0.5, 0.8, 0.2, 0.4};
    double y_vals[5] = {0.1, 0.2, 0.3, 0.7, 0.4};

    printf("-----\n");
    printf("1. E1(RGB, P_T):\n\n");

    printf(" ");
    for (int xi = 0; xi < 5; xi++) {

```



```

    printf("  x%d=%0.2f  ", xi + 1, x_vals[xi]);
}
printf("\n");

for (int yi = 0; yi < 5; yi++) {
    printf("y%d=%0.2f  ", yi + 1, y_vals[yi]);
    for (int xi = 0; xi < 5; xi++) {
        double x = x_vals[xi];
        double y = y_vals[yi];
        RGBIntensity E = {0.0, 0.0, 0.0};
        // if (x >= 0 && y >= 0 && (x + y) <= 1.0) {
            E = calculate_E_RGB_PT(
                P0, P1, P2,
                x, y,
                I0_1_RGB,
                PL1, O1_spot
            );
        // }
        printf("(%.3f,%.3f,%.3f) ", E.R, E.G, E.B);
    }
    printf("\n");
}

printf("-----
\n");
printf("2. E2(RGB, P_T):\n\n");

printf("
");
for (int xi = 0; xi < 5; xi++) {
    printf("  x%d=%0.2f  ", xi + 1, x_vals[xi]);
}
printf("\n");

for (int yi = 0; yi < 5; yi++) {
    printf("y%d=%0.2f  ", yi + 1, y_vals[yi]);
    for (int xi = 0; xi < 5; xi++) {
        double x = x_vals[xi];
        double y = y_vals[yi];
        RGBIntensity E = {0.0, 0.0, 0.0};
        // if (x >= 0 && y >= 0 && (x + y) <= 1.0) {
            E = calculate_E_RGB_PT(
                P0, P1, P2,
                x, y,
                I0_2_RGB,
                PL2, O2_spot
            );
        // }
        printf("(%.3f,%.3f,%.3f) ", E.R, E.G, E.B);
    }
    printf("\n");
}

printf("-----
\n");
printf("3. L(RGB, P_T, v):\n\n");

printf("
");
for (int xi = 0; xi < 5; xi++) {
    printf("  x%d=%0.2f  ", xi + 1, x_vals[xi]);
}
printf("\n");

```

```

for (int yi = 0; yi < 5; yi++) {
    printf("y%d=%0.2f ", yi + 1, y_vals[yi]);
    for (int xi = 0; xi < 5; xi++) {
        double x = x_vals[xi];
        double y = y_vals[yi];
        RGBIntensity L = {0.0, 0.0, 0.0};
        // if (x >= 0 && y >= 0 && (x + y) <= 1.0) {
            L = calculate_L_RGB_PT(
                P0, P1, P2,
                x, y,
                I0_1_RGB, I0_2_RGB,
                PL1, PL2,
                O1_spot, O2_spot,
                V_camera_static,
                K_RGB,
                kd, ks, ke
            );
        // }
        printf("(%.3f,%.3f,%.3f) ", L.R, L.G, L.B);
    }
    printf("\n");
}

printf("-----\n");

return 0;

```

Пример входных данных и результаты

1. $I_{0_1}(RGB) = (8, 6, 4)$
 $I_{0_2}(RGB) = (3, 5, 7)$
2. $\vec{O}_1 = (0.166, -0.643, -0.747)$
 $\vec{O}_2 = (-0.587, 0.151, -0.795)$
3. $\vec{P}_{L_1} = (0.20, 0.85, 0.60)$
 $\vec{P}_{L_2} = (0.85, 0.20, 0.70)$
4. $\vec{P}_0 = (0.0, 0.0, 0.0)$
5. $\vec{P}_1 = (1.0, 0.0, 0.0)$
6. $\vec{P}_2 = (0.0, 1.0, 0.0)$
7. $x_1 = 0.1, y_1 = 0.1$
 $x_2 = 0.5, y_2 = 0.2$
 $x_3 = 0.8, y_3 = 0.3$

$$x_4 = 0.2, y_4 = 0.7$$

$$x_5 = 0.4, y_5 = 0.4$$

$$8. \vec{V} = (0.0, 0.0, 1.0)$$

$$9. K(RGB) = (0.7, 0.7, 0.1)$$

$$10. k_d = 0.8$$

$$11. k_s = 0.2$$

$$12. k_e = 10.0$$

Вычисленные значения освещенности $E_1(RGB, \vec{P}_T)$ для точек, заданных локальными координатами:

$x \backslash y$	0,1	0,5	0,8	0,2	0,4
0,1	(5,047;3,785;2,524)	(4,592;3,444;2,296)	(3,008; 2,256; 1,504)	(5,251; 3,938; 2,625)	(4,996; 3,747; 2,498)
0,2	(6,496;4,872;3,248)	(5,778;4,334;2,889)	(3,553;2,665;1,777)	(6,793;5,095;3,397)	(6,384;4,788;3,192)
0,3	(8,338;6,254;4,169)	(7,233;5,417;3,611)	(0,000;0,000;0,000)	(8,774;6,580;4,387)	(8,126;6,095;4,063)
0,7	(16,459;12,344;8,230)	(0,000;0,000;0,000)	(0,000;0,000;0,000)	(17,876;13,407;8,938)	(0,000;0,000;0,000)
0,4	(10,563;7,923;5,282)	(8,881;6,661;4,440)	(0,000;0,000;0,000)	(11,194;8,396;5,597)	(10,196;7,647;5,098)

Вычисленные значения освещенности $E_2((RGB, \vec{P}_T))$ для тех же точек, заданных локальными координатами:

$x \backslash y$	0,1	0,5	0,8	0,2	0,4
------------------	-----	-----	-----	-----	-----

$y \backslash$					
0,1	(1,826;3,044;4,262)	(4,048;6,747;9,446)	(4,748; 7,913; 11,079)	(2,278; 3,797; 5,315)	(3,428; 5,714; 8,000)
0,2	(1,890;3,150;4,410)	(4,266;7,110;9,955)	(5,074;8,457;11,839)	(2,366;3,944;5,521)	(3,595;5,991;8,387)
0,3	(1,883;3,138;4,393)	(4,212;7,021;9,829)	(0,000;0,000;0,000)	(2,353;3,921;5,490)	(3,557;5,929;8,301)
0,7	(1,328;2,213;3,098)	(0,000;0,000;0,000)	(0,000;0,000;0,000)	(1,576;2,626;3,677)	(0,000;0,000;0,000)
0,4	(1,807;3,012;4,217)	(3,909;6,514;9,120)	(0,000;0,000;0,000)	(2,242;3,736;5,231)	(3,331;5,552;7,773)

Вычисленные значения яркостей $L(RGB, \vec{P}_T, \vec{v})$ для тех же точек на плоскости с заданными условиями наблюдения

$x \backslash y$	0,1	0,5	0,8	0,2	0,4
0,1	(1,479;1,478;0,211)	(1,993;2,435;0,411)	(1,923; 2,641; 0,480)	(1,639; 1,702; 0,252)	(1,895; 2,187; 0,354)
0,2	(1,842;1,764;0,241)	(2,333;2,746;0,451)	(2,147;2,902;0,523)	(2,032;2,017;0,286)	(2,270;2,509;0,392)
0,3	(2,308;2,110;0,273)	(2,668;2,979;0,470)	(0,000;0,000;0,000)	(2,532;2,388;0,321)	(2,693;2,814;0,419)
0,7	(4,699;3,783;0,410)	(0,000;0,000;0,000)	(0,000;0,000;0,000)	(5,190;4,205;0,460)	(0,000;0,000;0,000)
0,4	(2,892;2,532;0,310)	(3,005;3,147;0,470)	(0,000;0,000;0,000)	(3,163;2,835;0,358)	(3,178;3,118;0,437)

Вывод

В данной работе был изучен и реализован алгоритм вычисления яркости точки на плоскости с учетом освещенности от нескольких источников света. Используя заданные входные данные, программа корректно вычисляет значения освещенности и яркости для различных точек плоскости, учитывая направленность света и свойства поверхности.